Minería de Datos - Libro 1 - Clase 1 a 10

4. Visualizacion de Datos.html

Clase 4: Visualización de Datos

1. PRESENTACIÓN

Objetivo de la Clase

La visualización de datos es una herramienta fundamental en la minería de datos. No solo nos ayuda a entender los datos, sino que también facilita la comunicación de hallazgos y patrones a otras personas.

En esta clase, aprenderemos a:

- Identificar cuáles son los tipos de gráficos más adecuados para cada tipo de análisis.
- Usar Matplotlib y Seaborn para crear visualizaciones efectivas en Python.
- Aplicar técnicas avanzadas como pairplots y heatmaps para explorar relaciones entre múltiples variables.
- Interpretar gráficos correctamente para tomar mejores decisiones.

Importancia de la Visualización de Datos

- Permite comprender rápidamente grandes volúmenes de datos.
- Ayuda a detectar valores atípicos y errores en los datos.
- Facilita la identificación de patrones y tendencias.
- Mejora la presentación y comunicación de análisis a audiencias técnicas y no técnicas.

Ejemplo real: Un equipo de científicos de datos está analizando el comportamiento de clientes en una tienda en línea. A través de gráficos de dispersión y heatmaps, pueden visualizar qué productos suelen comprarse juntos y optimizar las recomendaciones.

1.1. ¿Por qué es importante la visualización de datos?

La visualización de datos es un paso clave en cualquier proceso de minería de datos. Permite comprender mejor los datos, detectar patrones, identificar valores atípicos y comunicar hallazgos de manera clara.

Beneficios de la visualización de datos:

- Facilita la comprensión de grandes volúmenes de información.
- Ayuda a detectar errores y valores atípicos en los datos.
- Permite identificar relaciones entre variables.
- Mejora la comunicación de resultados en equipos multidisciplinarios.

Ejemplo real: Un banco analiza datos de clientes para otorgar créditos. Un gráfico de dispersión entre ingresos y nivel de endeudamiento puede ayudar a visualizar patrones y tomar mejores decisiones.

1.2. Tipos de Visualización y Cuándo Usarlos

Cada tipo de gráfico tiene un propósito específico. La siguiente tabla resume qué gráfico usar según el análisis que queremos realizar.

Tipo de Análisis	Objetivo	Gráfico Recomendado
Distribución de una variable	Entender cómo se distribuyen los valores.	**Histograma, KDE Plot**
Valores atípicos en una variable	Detectar outliers o datos anómalos.	**Boxplot**
Relación entre dos variables numéricas	Ver si hay correlación o dependencia.	**Scatter Plot, Line Plot**
Comparación de categorías	Comparar valores de diferentes grupos.	**Bar Chart, Violin Plot**
Evolución de datos en el tiempo	Identificar tendencias en series temporales.	**Line Plot**
Relación entre múltiples variables	Ver interacciones en varios ejes.	**Pairplot, Heatmap**

Ejemplo en Python – Diferentes Tipos de Visualización

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear datos ficticios
df = pd.DataFrame({
    "Edad":      [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos":  [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000],
    "Categoría": ["Joven", "Adulto", "Adulto", "Adulto", "Senior", "Senior", "Joven", "Adulto", "Adulto", "Senior"]
})

# Histograma de edades
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

# Boxplot de ingresos
sns.boxplot(x=df["Ingresos"])
plt.title("Boxplot de Ingresos")
plt.show()

# Scatter Plot entre Edad e Ingresos
sns.scatterplot(x=df["Edad"], y=df["Ingresos"], hue=df["Categoría"])
plt.title("Relación entre Edad e Ingresos")
plt.show()
```

Conclusión:

- El histograma nos permite ver la distribución de edades.
- El boxplot ayuda a detectar posibles valores atípicos en ingresos.
- El scatter plot muestra la relación entre Edad e Ingresos, diferenciando categorías.

Ejercicio para la Sección 1

Objetivo: Practicar la selección de gráficos según el tipo de análisis.

Instrucciones:

1. Selecciona el gráfico adecuado para cada uno de los siguientes análisis:
    - Comparar el salario promedio en diferentes industrias.
    - Analizar si existe una relación entre la cantidad de horas trabajadas y el nivel de productividad.
    - Detectar valores atípicos en los montos de transacciones bancarias.
    - Observar la distribución de notas de un curso.
2. Explica por qué elegiste cada tipo de gráfico.
3. Si tienes conocimientos en Python, escribe un código utilizando Seaborn para visualizar alguno de los casos.

Sección 2: Matplotlib y Seaborn

2.1. ¿Qué es Matplotlib y cómo usarlo?

Matplotlib es la biblioteca más básica y flexible para crear gráficos en Python. Se usa principalmente para gráficos simples como líneas, barras y dispersión.

Ejemplo en Python – Crear un gráfico de líneas con Matplotlib

```
import matplotlib.pyplot as plt

# Datos de ejemplo
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 50]

# Crear gráfico de líneas
plt.plot(x, y, marker='o', linestyle='-', color='b', label="Crecimiento")
plt.xlabel("Tiempo")
plt.ylabel("Valor")
plt.title("Ejemplo de Gráfico de Líneas")
plt.legend()
plt.show()
```

Conclusión: Matplotlib permite personalizar cada aspecto del gráfico, como colores, etiquetas y estilos de líneas.

2.2. ¿Qué es Seaborn y cómo usarlo?

Seaborn está basado en Matplotlib, pero ofrece una sintaxis más sencilla y gráficos más atractivos por defecto. Se usa para visualizaciones estadísticas avanzadas, como distribuciones de datos, correlaciones y visualización de múltiples variables.

Ejemplo en Python – Histogramas y Boxplots con Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Crear un DataFrame con datos ficticios
df = pd.DataFrame({
    "Edad":     [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos": [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000]
})

# Histograma de edades
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

# Boxplot de ingresos
sns.boxplot(x=df["Ingresos"])
plt.title("Boxplot de Ingresos")
plt.show()
```

Conclusión: Seaborn es ideal para gráficos estadísticos más complejos, como histogramas con KDE, boxplots y pairplots.

2.3. Comparación entre Matplotlib y Seaborn

Característica	Matplotlib	Seaborn
Facilidad de uso	Más flexible pero requiere más código.	Más intuitivo y automático.
Personalización	Muy alta (permite modificar cada detalle).	Alta, pero con menos código necesario.
Aplicaciones	Gráficos básicos y personalizables.	Gráficos estadísticos más avanzados.
Ejemplo típico	Gráficos de líneas y barras.	Histogramas, heatmaps y pairplots.

¿Cuándo usar cada uno?

- Usa **Matplotlib** cuando necesites total control sobre los detalles del gráfico.
- Usa **Seaborn** cuando necesites gráficos estadísticos listos para usar con menos código.

Ejercicio para la Sección 2

Objetivo: Practicar el uso de Matplotlib y Seaborn creando gráficos de distintos tipos.

Instrucciones:

1. Usa Matplotlib para crear un gráfico de barras que muestre el promedio de ingresos de diferentes categorías de empleados.
2. Usa Seaborn para crear un scatterplot que muestre la relación entre Edad e Ingresos en un dataset de empleados.
3. Explica cuál de los dos gráficos es más fácil de interpretar y por qué.

Pista para la solución en Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear DataFrame
df = pd.DataFrame({
    "Categoría":        ["Junior", "Semi-Senior", "Senior"],
    "Salario_Promedio": [35000, 60000, 90000]
})

# Gráfico de barras con Matplotlib
plt.bar(df["Categoría"], df["Salario_Promedio"], color=["blue", "green", "red"])
plt.xlabel("Categoría de Empleado")
plt.ylabel("Salario Promedio")
plt.title("Salario Promedio por Categoría")
plt.show()

# Crear DataFrame con edades e ingresos
df2 = pd.DataFrame({
    "Edad":     [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos": [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000]
})

# Scatterplot con Seaborn
sns.scatterplot(x=df2["Edad"], y=df2["Ingresos"])
plt.title("Relación entre Edad e Ingresos")
plt.show()
```

Preguntas de Reflexión

1. ¿Cuál de los dos gráficos te parece más útil para analizar la información?
2. ¿Cómo podría mejorarse la interpretación de estos gráficos?
3. ¿Cómo podrías aplicar esto en un dataset real?

Sección 3: Gráficos Avanzados con Seaborn (Pairplot, Heatmaps y Violin Plots)

Objetivo: Aprender a utilizar gráficos avanzados en Seaborn para visualizar relaciones complejas en los datos.

3.1. Pairplot: Exploración Multivariable

¿Qué es un Pairplot?
Es una herramienta que muestra relaciones entre múltiples variables en un dataset de manera eficiente.

¿Cuándo usarlo?

- Cuando queremos ver cómo se relacionan múltiples variables numéricas en un solo gráfico.
- Cuando queremos detectar correlaciones visualmente sin calcular matrices de correlación.

Ejemplo en Python – Pairplot con Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Cargar dataset de pingüinos de Seaborn
df = sns.load_dataset("penguins")

# Crear Pairplot
sns.pairplot(df, hue="species", diag_kind="kde")
plt.show()
```

Conclusión:

- Cada gráfico en la diagonal muestra la distribución univariada de cada variable.
- Los gráficos fuera de la diagonal muestran relaciones entre pares de variables.

3.2. Heatmaps: Mapas de Correlación

¿Qué es un Heatmap?
Es un mapa de calor que muestra cómo se correlacionan las variables numéricas en un dataset.

¿Cuándo usarlo?

- Cuando queremos analizar fuerza y dirección de correlación entre variables.

- Cuando buscamos variables redundantes que pueden eliminarse o agruparse.

Ejemplo en Python – Heatmap de correlaciones

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Cargar dataset
df = sns.load_dataset("penguins").dropna()

# Calcular matriz de correlación
correlaciones = df.corr(numeric_only=True)

# Crear Heatmap
sns.heatmap(correlaciones, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Mapa de Correlación entre Variables")
plt.show()
```

Conclusión:

- Un coeficiente cercano a **1** indica alta correlación positiva (ejemplo: peso y tamaño).
- Un coeficiente cercano a **-1** indica alta correlación negativa (ejemplo: velocidad y tiempo de viaje).
- Un coeficiente cercano a **0** significa que las variables no están relacionadas.

3.3. Violin Plots: Comparación de Distribuciones

¿Qué es un Violin Plot?
Es una mezcla entre un boxplot y un histograma, que muestra la distribución de los datos dentro de cada categoría.

¿Cuándo usarlo?

- Cuando queremos ver cómo varían los valores en diferentes grupos.
- Cuando queremos detectar asimetrías en la distribución de datos.

Ejemplo en Python – Violin Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

# Cargar dataset
df = sns.load_dataset("penguins")

# Crear Violin Plot
sns.violinplot(x="species", y="bill_length_mm", data=df, palette="muted")
plt.title("Distribución del Largo del Pico por Especie")
plt.show()
```

Conclusión:

- El ancho del violín representa dónde se encuentran más datos.
- Las colas indican valores extremos o atípicos.

Ejercicio para la Sección 3

Objetivo: Aplicar gráficos avanzados para extraer insights de un dataset.

Instrucciones:

1. Carga el dataset de diamantes de Seaborn (`diamonds`).
2. Crea un Pairplot para analizar la relación entre precio, quilates y profundidad.
3. Genera un Heatmap para visualizar la correlación entre las variables numéricas.
4. Crea un Violin Plot para comparar la distribución del precio en cada categoría de corte (`cut`).
5. Escribe una conclusión basada en los gráficos.

Pista para la solución en Python

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Cargar dataset
df = sns.load_dataset("diamonds")

# 1. Pairplot entre precio, quilates y profundidad
sns.pairplot(df[["price", "carat", "depth"]])
plt.show()

# 2. Heatmap de correlaciones
correlaciones = df.corr(numeric_only=True)
sns.heatmap(correlaciones, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Mapa de Correlación de Diamantes")
plt.show()

# 3. Violin Plot para comparar precio según corte
sns.violinplot(x="cut", y="price", data=df, palette="muted")
plt.title("Distribución del Precio por Tipo de Corte")
plt.show()
```

Preguntas de Reflexión

1. ¿Qué variables están más correlacionadas con el precio?
2. ¿Cómo varía la distribución del precio en los diferentes tipos de corte?
3. ¿Qué otros insights puedes extraer de estos gráficos?

Actividad Obligatoria

Entregarla en el espacio para la misma dentro del Campus.

Objetivo: Aplicar todas las técnicas de visualización de datos vistas en la clase para analizar un dataset real y compartir los hallazgos en el foro.

4.1. Ejercicio Integrador

Supongamos que estamos analizando un conjunto de datos de ventas de una tienda en línea. Queremos entender el comportamiento de los clientes y visualizar patrones clave que puedan ayudar a mejorar la estrategia de ventas.

Dataset para el ejercicio

Cliente	Edad	Ingresos	Compras	Categoría
1	25	50000	5	A
2	40	60000	15	B
3	35	70000	10	C
4	50	80000	20	A
5	30	45000	7	B
6	45	85000	18	C
7	38	72000	12	A
8	29	48000	6	B

Tareas a realizar

1. **Análisis Exploratorio de Datos (EDA)**
   - Generar histogramas para Edad e Ingresos.
   - Crear un boxplot para visualizar valores atípicos en Compras.
   - Generar un scatter plot para analizar la relación entre Ingresos y Compras.
2. **Visualización Avanzada**
   - Crear un Pairplot con las variables numéricas.
   - Generar un Heatmap de correlaciones entre todas las variables numéricas.
   - Utilizar un Violin Plot para comparar la distribución de Compras según la Categoría.
3. **Interacción con una Herramienta de IA (LLM)**
   - Hacer una consulta a ChatGPT, Gemini u otra herramienta LLM para obtener insights sobre los datos.

- Ejemplo de consulta: "*Tengo un dataset con datos de clientes (edad, ingresos y número de compras). ¿Qué visualizaciones me recomendarías para encontrar patrones?*"
- Comparar la respuesta con lo aprendido en la clase y reflexionar sobre su utilidad.

Código Base para la Solución

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear DataFrame
df = pd.DataFrame({
    "Cliente":   [1, 2, 3, 4, 5, 6, 7, 8],
    "Edad":      [25, 40, 35, 50, 30, 45, 38, 29],
    "Ingresos":  [50000, 60000, 70000, 80000, 45000, 85000, 72000, 48000],
    "Compras":   [5, 15, 10, 20, 7, 18, 12, 6],
    "Categoría": ["A", "B", "C", "A", "B", "C", "A", "B"]
})

# 1) Análisis Exploratorio
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

sns.boxplot(x=df["Compras"])
plt.title("Boxplot de Compras")
plt.show()

sns.scatterplot(x=df["Ingresos"], y=df["Compras"])
plt.title("Relación entre Ingresos y Compras")
plt.show()

# 2) Visualización Avanzada
sns.pairplot(df[["Edad", "Ingresos", "Compras"]])
plt.show()

# Mapa de calor de correlaciones
correlaciones = df.corr(numeric_only=True)
sns.heatmap(correlaciones, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Mapa de Correlación de Variables")
plt.show()

# Violin Plot de Compras por Categoría
sns.violinplot(x="Categoría", y="Compras", data=df, palette="muted")
plt.title("Distribución de Compras por Categoría")
plt.show()
```

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Elsevier.
2. Wickham, H., & Grolemund, G. (2016). *R for Data Science*. O'Reilly Media.
3. McKinney, W. (2017). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
4. Seaborn Documentation. (n.d.). *Seaborn: Statistical Data Visualization*. Recuperado de https://seaborn.pydata.org/
5. Matplotlib Developers. (n.d.). *Matplotlib: Visualization with Python*. Recuperado de https://matplotlib.org/
6. Tufte, E. R. (2001). *The Visual Display of Quantitative Information*. Graphics Press.
7. ChatGPT & Gemini (2023). *Uso de Modelos de Lenguaje para la Exploración de Datos*. Recuperado de https://openai.com